Jenkins Mastery

Build Pipeline

Visualising · Chaining · Parallelising Jenkins Jobs

Saleem Ahmed
linkedin.com/in/saleemfai · Cameroon
Jenkins 2.541.2 · April 2026

Table of Contents

1. Overview — What is a Build Pipeline?
2. Phase 1 — Install Required Plugins
3. Phase 2 — Create Upstream & Downstream Jobs
4. Phase 3 — Configure the Pipeline View
5. Phase 4 — Run the Basic Pipeline
6. Phase 5 — Continuous Delivery (Multi-Stage Pipeline)
7. Phase 6 — Parallel Jobs in a Pipeline
8. Key Lessons Learned

1. Overview — What is a Build Pipeline?

A **Build Pipeline** in Jenkins is a visual representation of a sequence of jobs chained together — where the successful completion of one job automatically triggers the next. Instead of managing individual jobs in isolation, a pipeline gives you an end-to-end view of the entire delivery process: from build, through test, all the way to deployment.

The **Build Pipeline Plugin** extends Jenkins with a dedicated pipeline view that shows each job as a stage card, the flow of execution between them with directional arrows, and real-time build status using colour-coded indicators.

Colour Coding at a Glance

Colour	Meaning	Action Required
Green	SUCCESS — stage completed	None — proceed
Yellow	IN PROGRESS — stage running	Wait for completion
Blue/Grey	NOT YET RUN — pending	Will trigger automatically
Red	FAILED — stage errored	Investigate and fix

2. Phase 1 — Install the Required Plugins

The Build Pipeline view requires two plugins that are not bundled with Jenkins by default. Both must be installed before creating pipeline views or chaining jobs with parameterized triggers.

Plugins Required

Plugin	Purpose
Build Pipeline	Adds the pipeline view and stage card visualisation
Parameterized Trigger	Allows upstream jobs to pass parameters downstream

1 Open the Plugin Manager

From the Jenkins Dashboard, navigate to:

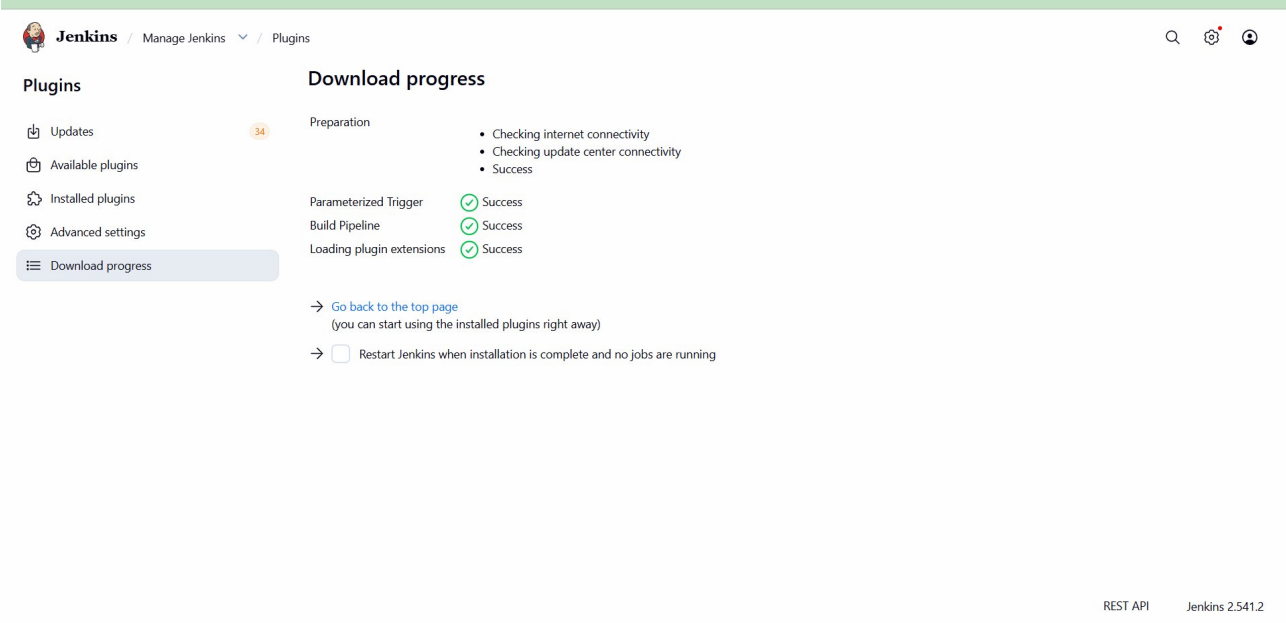
```
Manage Jenkins → Plugins → Available plugins
```

2 Search, Select, and Install Both Plugins

Search for **Build Pipeline** and **Parameterized Trigger**, check both, then click **Install**.

3 Verify the Installation

On the Download progress page, confirm that both plugins show a green ■ Success status alongside **Loading plugin extensions**. No Jenkins restart is required.



■ *Plugin download progress — Parameterized Trigger and Build Pipeline both showing Success*

Note: You can start using installed plugins immediately without restarting Jenkins. The page confirms this with the 'Go back to the top page' link.

3. Phase 2 — Create the Upstream and Downstream Jobs

A pipeline requires at least two jobs connected by a Post-build Action — an **upstream** job that runs first and triggers, and a **downstream** job that is triggered. The connection lives entirely inside the upstream job's configuration.

Step 4 — Create the Upstream Job: project-build

1 Create a new Freestyle job

```
Jenkins Dashboard → New Item → project-build → Freestyle Project → OK
```

2 Add a Build Step

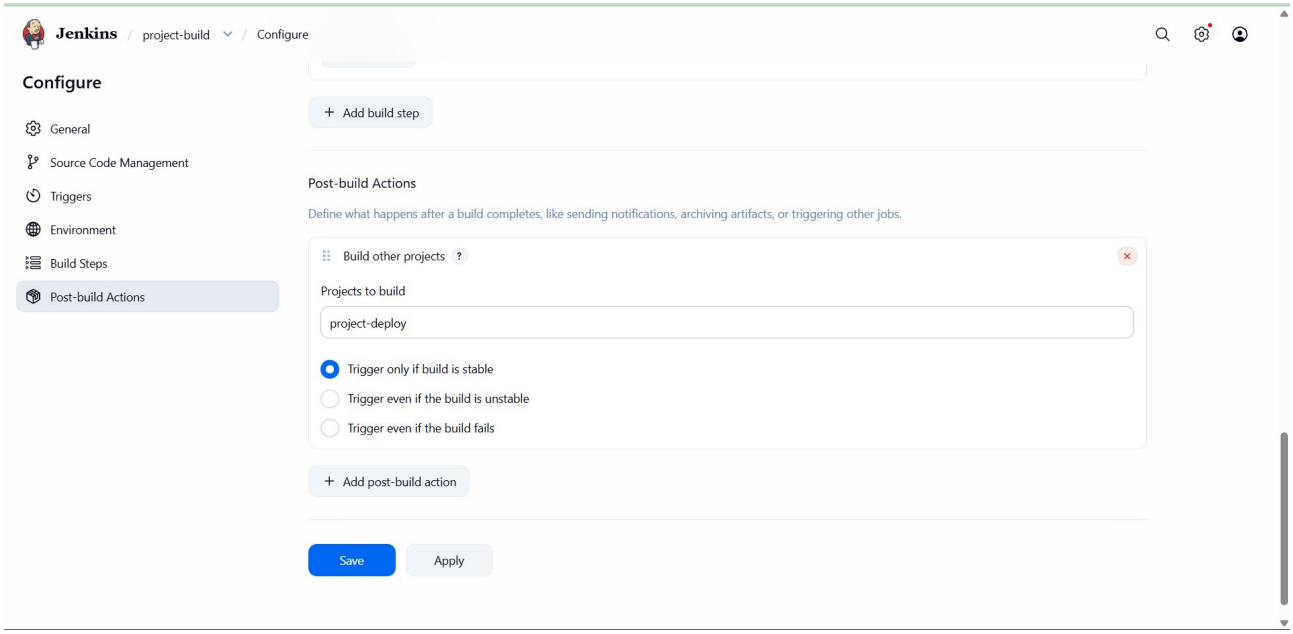
Under **Build Steps** → **Execute shell**, add:

```
echo "Building the project..."
sleep 10
echo "Build complete."
```

3 Configure Post-build Action to chain jobs

Scroll to **Post-build Actions** and click **Add post-build action** → **Build other projects**.

Field	Value	Purpose
Projects to build	project-deploy	The job to trigger downstream
Trigger condition	Trigger only if build is stable	Only advance on SUCCESS



project-build Post-build Actions — Build other projects targeting project-deploy, trigger only if build is stable

Step 5 — Create the Downstream Job: project-deploy

1 Create a second Freestyle job

```
Jenkins Dashboard → New Item → project-deploy → Freestyle Project → OK
```

2 Add a Build Step

```
echo "Deploying the project..."
sleep 10
echo "Deployment complete."
```

Click **Save**. No Post-build Action is needed on the last stage — it is the end of the chain.

4. Phase 3 — Configure the Pipeline View

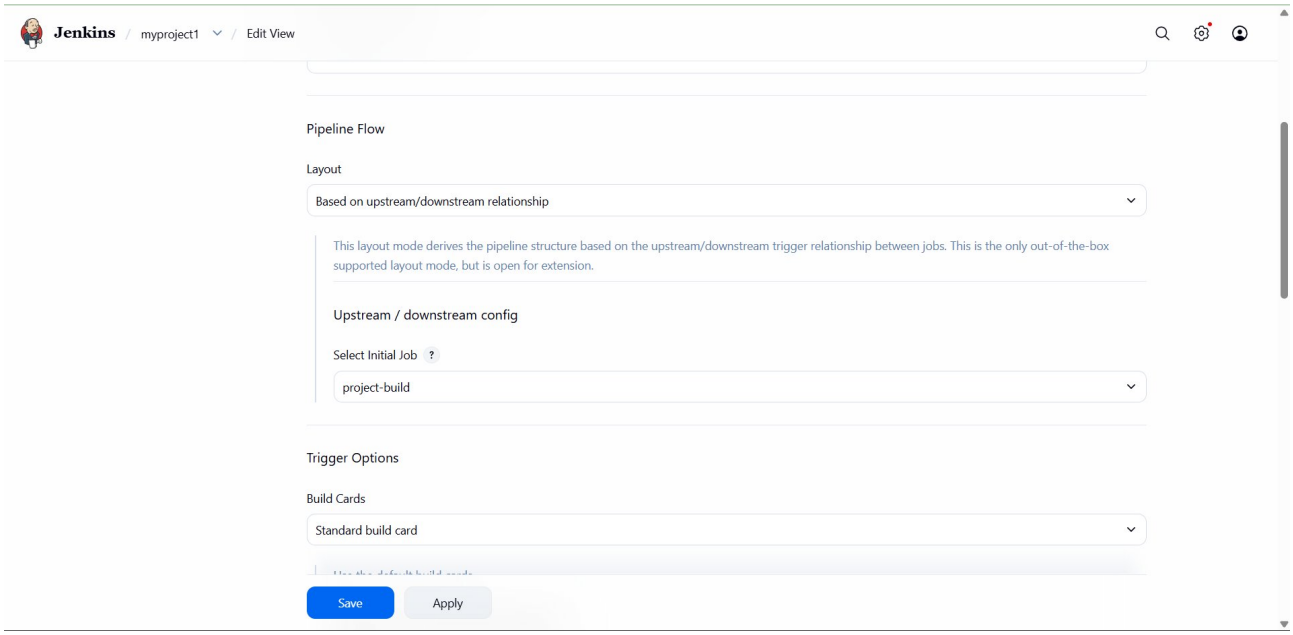
With the jobs created and linked, the next step is to create a dedicated pipeline view. The view reads the existing upstream/downstream relationships and renders them as stage cards — no re-configuration of the jobs is required.

1 Create a new view from the Dashboard

```
Jenkins Dashboard → + (New View) → View name: myproject1 → Build Pipeline View → OK
```

2 Configure Pipeline Flow settings

Field	Value	Explanation
Layout	Based on upstream/downstream relationship	Auto-discovers chain from trigger links
Select Initial Job	project-build	The first job — anchors the view
Build Cards	Standard build card	Default card style with metadata



Pipeline view Edit View — Pipeline Flow section configured with upstream/downstream layout and project-build as the initial job

3 Save the view

Click **Save**. Jenkins renders the pipeline view immediately, showing all discovered stages as cards connected by arrows.

5. Phase 4 — Run the Basic Pipeline

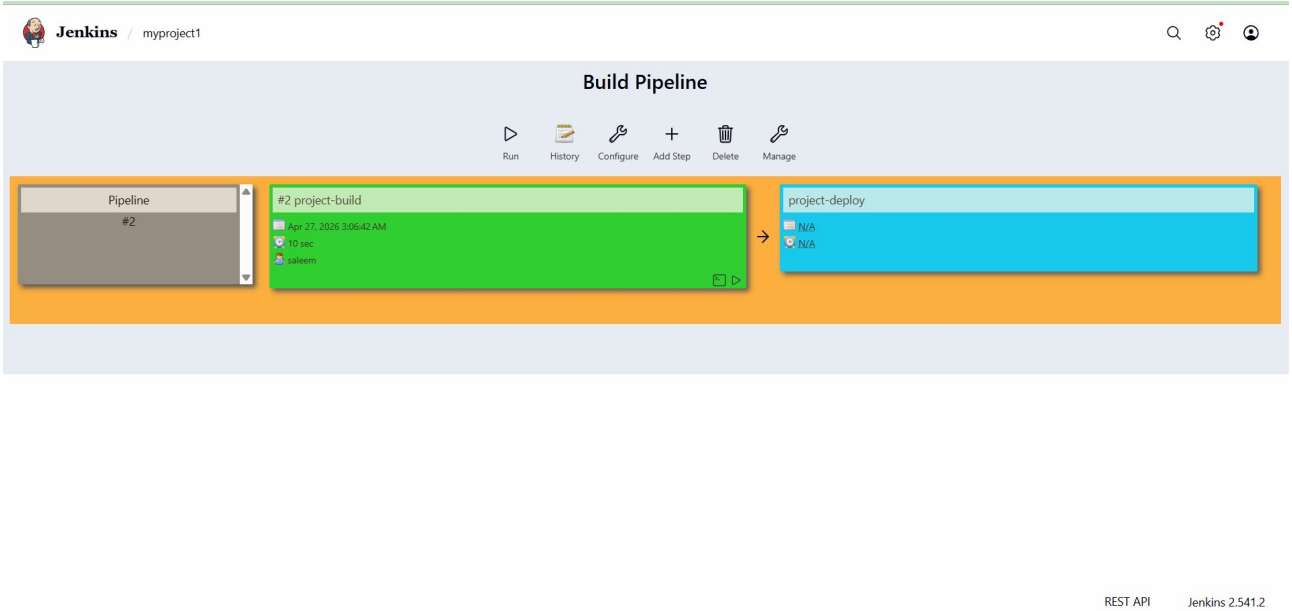
With the view configured, triggering a run from inside the pipeline view starts the upstream job and lets Jenkins manage the rest of the chain automatically.

1 Click Run in the myproject1 view

Jenkins starts **project-build** (Build #2). The stage card turns yellow and shows a progress indicator.

2 Watch the automatic handoff

When **project-build** completes successfully, Jenkins automatically triggers **project-deploy** — no manual action required.



myproject1 Build Pipeline view — project-build #2 (green/SUCCESS), arrow pointing to project-deploy (cyan/queued for trigger)

Result: The pipeline view showed both stages side by side. project-build completed in 10 seconds and automatically handed execution to project-deploy — the entire delivery chain ran without any manual steps between stages.

> **Note:** The cyan/light-blue colour on project-deploy indicates the stage has been queued and is waiting to start. It turns green once it completes successfully.

6. Phase 5 — Continuous Delivery Pipeline (Multi-Stage)

To simulate a real-world Continuous Delivery workflow, a three-stage pipeline was built using three chained jobs: **test1** → **staging** → **prod**. Each stage represents a promotion gate — code only advances to the next environment when the current stage succeeds.

Three-Stage Job Structure

Job	Environment Role	Triggers
test1	Test environment — validates the build	Triggers staging on success
staging	Staging environment — pre-production check	Triggers prod on success
prod	Production environment — live deployment	End of pipeline

1 Create test1, staging, and prod jobs

Each job is a Freestyle job with an Execute shell build step and a Post-build Action linking to the next job.

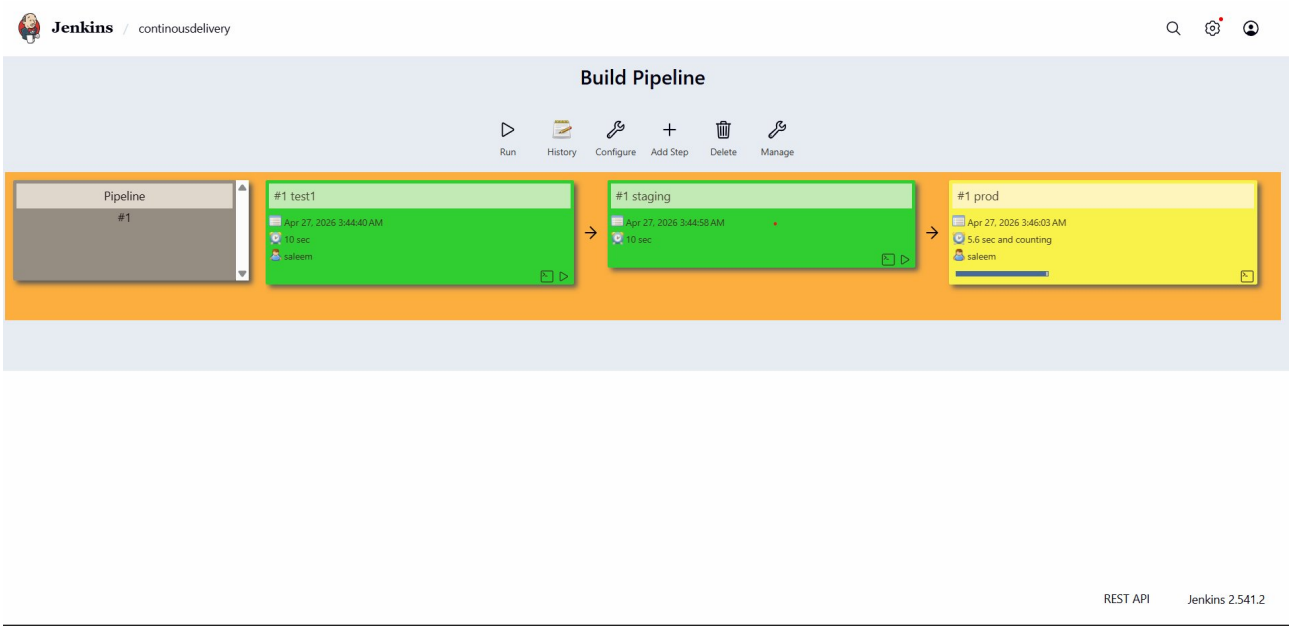
```
# Post-build Action pattern for test1:
Add post-build action → Build other projects
Projects to build: staging
Trigger only if build is stable  ← selected

# Post-build Action pattern for staging:
Add post-build action → Build other projects
Projects to build: prod
Trigger only if build is stable  ← selected
```

2 Create the continuousdelivery pipeline view

Create a new Build Pipeline View called **continuousdelivery** with **test1** set as the Initial Job.

3 Trigger the pipeline and observe live propagation

Click **Run**. Jenkins executes test1 → staging → prod sequentially. The view shows live status for each stage.



■ *continuousdelivery Build Pipeline — test1 #1 (green), staging #1 (green), prod #1 (yellow/in-progress with progress bar visible)*

Result: The three-stage pipeline ran end-to-end. Green cards confirmed completed stages while the yellow progress bar on prod showed real-time execution. Code flowed automatically from test to staging to production with no manual promotion steps.

7. Phase 6 — Parallel Jobs in a Pipeline

Jenkins Build Pipeline also supports **parallel execution** — where multiple downstream jobs are triggered from a single upstream job simultaneously. This is ideal when two independent tasks (such as deploying to two environments at the same time) do not need to wait for each other.

How Parallel Triggering Works

To trigger multiple jobs in parallel, simply list them as comma-separated values in the **Projects to build** field of the Post-build Action. Jenkins will fire all listed jobs simultaneously when the upstream build succeeds.

1 Update test1's Post-build Action for parallel triggering

Inside **test1** → **Configure** → **Post-build Actions** → **Build other projects**, update the Projects to build field:

```
staging, staging2
```

With **Trigger only if build is stable** selected, both **staging** and **staging2** will be triggered simultaneously whenever test1 succeeds.

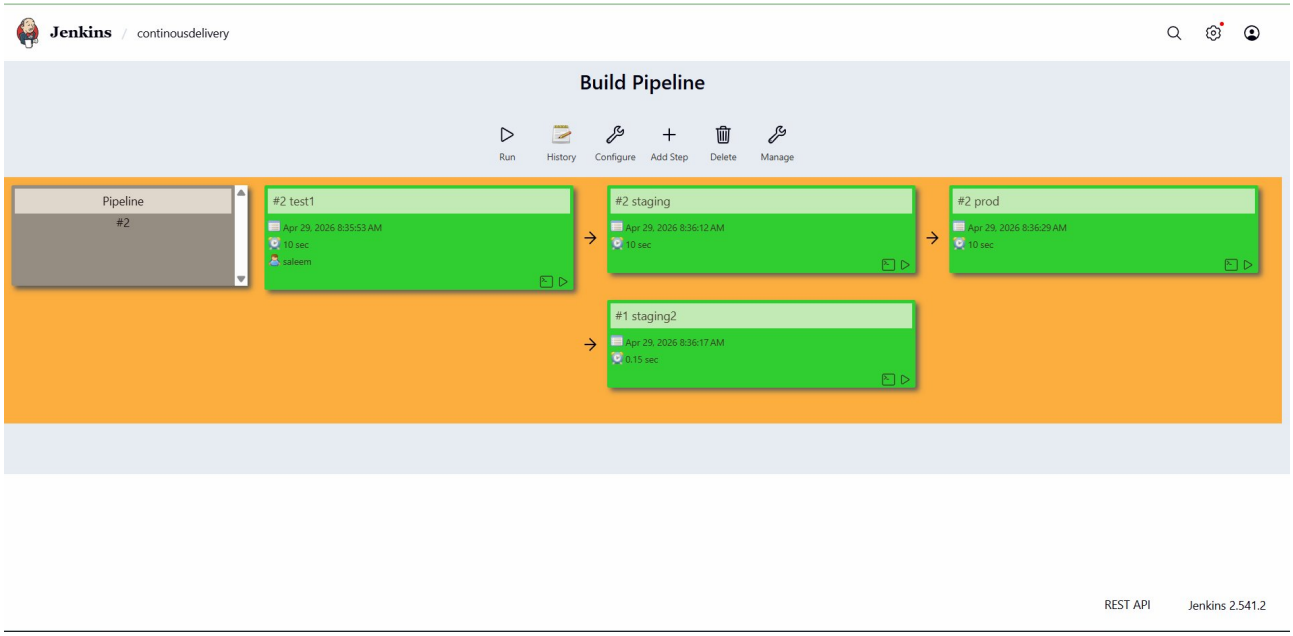
2 Create the staging2 job

Create a new Freestyle job called **staging2** with a simple shell build step. No Post-build Action is needed if staging2 is a terminal stage.

```
echo "Running parallel staging checks..."
sleep 10
echo "Parallel staging complete."
```

3 Trigger the pipeline and observe parallel execution

Click **Run** in the continuousdelivery view. test1 starts and upon success, both staging and staging2 are triggered at the same time.



continuousdelivery Build Pipeline #2 — test1 (green) branching into staging and staging2 running in parallel (both green), feeding into prod (green, complete)

Result: The pipeline view rendered the parallel branches clearly — test1 triggering two separate stage cards side by side. Both staging and staging2 completed before prod was triggered. This confirmed that Jenkins correctly handles fan-out parallel execution in a Build Pipeline view.

> **Note:** Parallel jobs share the available executors. If only one executor is free, the second parallel job will queue until an executor becomes available. Configure enough executors on your Jenkins agents for true parallel execution.

8. Key Lessons Learned

The following principles summarise the core insights from working with the Jenkins Build Pipeline plugin across all phases of this lab.

1. The Build Pipeline Plugin Makes Job Chains Visible

Without the plugin, upstream/downstream relationships are invisible — you only know a job triggered another by reading logs. The Build Pipeline view makes the entire chain visible at a glance, with colour-coded status per stage and directional arrows showing execution flow.

2. Post-build Actions Are the Glue Between Stages

The connection between two jobs in a pipeline is always defined in the upstream job's Post-build Actions section using Build other projects. There is no special pipeline-level configuration — the view simply reads the existing trigger relationships.

3. 'Trigger Only If Build Is Stable' Is the Right Default

Choosing this option ensures that a failure in any stage stops the pipeline from advancing. Code that fails tests does not reach staging. Code that fails staging does not reach production. This is the foundation of safe, automated delivery.

4. The Initial Job Anchors the Entire View

The pipeline view derives its structure by following the trigger chain starting from the Initial Job. Set this to the very first job in your delivery sequence — everything downstream is discovered automatically. Changing it changes the entire view.

5. Parallel Branches Maximise Throughput

When two downstream tasks are independent of each other, there is no reason to run them sequentially. Listing multiple jobs in the Build other projects field (comma-separated) triggers them all at once — cutting total pipeline time when executors are available.

6. Colour Coding Gives Instant Build Health Feedback

Green = SUCCESS, Yellow = IN PROGRESS, Blue/Grey = NOT YET RUN. A single look at the pipeline view tells you exactly where the pipeline stands without opening a single job log. This is invaluable during incident response.

7. Pipeline Views Are Read-Only Visualisations

The Build Pipeline view does not change how jobs work — it only visualises the upstream/downstream relationships that already exist. You can delete the view without affecting any job or trigger configuration.

Summary — What Was Built

Phase	What Was Done	Result
1 — Plugin Install	Installed Build Pipeline + Parameterized Trigger	Both plugins active, no restart needed
2 — Job Creation	Created project-build and project-deploy with Post-build chaining	Two-stage pipeline operational
3 — Pipeline View	Created myproject1 view with upstream/downstream layout	Visual pipeline dashboard ready
4 — Basic Run	Triggered pipeline — automatic handoff confirmed	End-to-end delivery in one click
5 — CD Pipeline	Built test1 → staging → prod continuous delivery chain	Three-stage promotion gate operational
6 — Parallel Jobs	Configured test1 to trigger staging + staging2 simultaneously	Fan-out parallel execution confirmed

Jenkins Mastery · Saleem Ahmed · linkedin.com/in/saleemfai · Cameroon